
Problem 1

Write a regex finding in a text as in the example below only words beginning with a capital letter and enclosed in single apostrophes. Do *not* use any methods of class **String**. For example, the program of the following scheme:

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;
public class RegTest {
    public static void main(String[] args) {
        String text = "'Angola', Germany, 'togo', 'Poland'";
        String reg = /* ??? */
        Matcher m = /* ??? */
        while (m.find()) System.out.println(/* ??? */);
    }
}
```

should print

```
Angola
Poland
```

Problem 2

Write a regex finding in a text as in the example below only the *second* part of hyphenated words; Do *not* use any methods of class **String**. For example, the program of the following scheme:

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;
public class RegTest {
    public static void main(String[] args) {
        String text =
            "one-half, -10.0 - mother-in-low dry-cleaning";
        String reg = /* ??? */
        Matcher m = /* ??? */
        while (m.find()) System.out.println(/* ??? */);
    }
}
```

should print

```
half
in
cleaning
```

Problem 3

Write a regex finding in a text at least three-letter words in such a way that the program as below prints only the first and the last letters of these words, separated with a hyphen. Do *not* use any methods of class **String**. For example, the program of the following scheme:

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;
public class RegTest {
    public static void main(String[] args) {
        String text =
            "to be, or not to be, that is the question";
        String reg = /* ??? */
        Matcher m = /* ??? */
        while (m.find()) System.out.println(/* ??? */);
    }
}
```

should print

```
n-t
t-t
t-e
q-n
```

Problem 4

We assume that the correct date format is: day (one or two digits), month (one or two digits) and year (exactly four digits). The day and month number and month and year number are separated by *the same* separator which can *only* be a slash, a dash or a dot.

Complete the following program so it replaces in a given text all occurrences of correct dates by the same dates but in the year-month-day format with the same separator as in the original form:

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class DayMonthYear {
    public static void main (String[] args) {
        String text = "good 2-01-2020 abc \nwrong " +
            "4/7-2010\ngood 11/11/2011 " +
            "wrong 211/3/1998 jkl\n" +
            "good 13.9.1999 wrong 1.1.20121\n" +
            "good 11.05.1823, wrong 9_10_2000";
        System.out.println("*** Before\n" + text);
        String r = /* regex */;
        String s = /* replacement */;
```

[download DayMonthYear.java](#)

```

        text = text.replaceAll(r, s);
        System.out.println("\n*** After\n" + text);
    }
}

```

For the text as given in the example above, the result should be

```

*** Before
good 2-01-2020 abc
wrong 4/7-2010
good 11/11/2011 wrong 211/3/1998 jkl
good 13.9.1999 wrong 1.1.20121
good 11.05.1823, wrong 9_10_2000

*** After
good 2020-01-2 abc
wrong 4/7-2010
good 2011/11/11 wrong 211/3/1998 jkl
good 1999.9.13 wrong 1.1.20121
good 1823.05.11, wrong 9_10_2000

```

Problem 5

In document preparation system L^AT_EX, widely used for typesetting texts, especially scientific and technical, commands have usually the form `\command{argument}`, where `argument` may contain several words. In particular, `\index{words}` inserts `words` as an entry of the index at the end of the document.

Write a program which reads a L^AT_EX file and, using regular expressions, finds all entries inside the `\index{...}` commands and writing these entries to a text file, each entry on a separate line. Note that an entry may contain several words and non-alphanumeric characters. It can also happen that the first part of an entry is at the end of one line, while its continuation is at the beginning of the next one.

For example, for the file containing a Polish text encoded in UTF-8 (do not change its division into lines)

```

Za początek okresu hellenistycznego uznaje się śmierć
\emph{Stagiryty}.\index{Stagiryta} To okres powstania
do dziś żywych koncepcji etycznych: \emph{epikureizmu}
(Epikur,\index{Epikur} ur. 341, zm. 270 p.n.e.)
mówiącego, że sensem życia jest doznawanie przyjemności
(por. \emph{hedonizm}) i \emph{stoicyzmu} (IV wiek p.n.e.
- II n.e.) dającego wzór postawy mędrca wobec śmierci
(Zenon z Kition,\index{Zenon
z Kition} Chryzyp, Seneka, mistrz retoryki Cyceon
i Marek Aureliusz\index{Marek
Aureliusz}, filozof na tronie). Sceptycy starożytni (Pirron,
\index{Pirron} Sekstus Empiryk)\index{Sekstus

```

Empiryk} z pobudek etycznych (uspokojenie umysłu) podawali
w wątpliwość możliwość wszelkiej wiedzy i dali początek wszelkim
nurtom sceptycznym. Tomasz z Akwinu\index{Tomasz
z Akwinu} (ur. 1225, zm. 1274)

the result should be

Stagiryta
Epikur
Zenon z Kitium
Marek Aureliusz
Pirron
Sekstus Empiryk
Tomasz z Akwinu
